NextIn: A Virtual Queue App

Project Proposal (CS 501 E1)

Team: Namratha Reddy (U65966062) and Suchir Jolapara (U08469805)

Repository: https://github.com/namz0516/CS501-NextIn

Platform: Android mobile application and a Wear OS app, built with Kotlin and Jetpack Compose

1. Project Concept

Description: NextIn is an Android app that lets customers join a business's waiting list from their phone, see their place in line and estimated wait, and leave the waiting area without losing their spot. Business staff use a separate dashboard to manage the same list, including walk-in parties who don't have the app. Customers can also check their status on a Wear OS smartwatch, and the app suggests nearby businesses if the wait is too long

Problem: Walk-in customers at places like restaurants, salons, and clinics usually have no idea how long they will wait. They either stay crowded near the front desk or keep asking staff for updates. Staff also have a harder job than a normal line: they have to keep track of everyone waiting, and at restaurants they also have to decide which party fits the tables that just opened up, often with a paper list

Why a mobile app: People already carry their phones, so a virtual queue lets them wait at a nearby cafe or in the car and still see live updates. A watch makes this easier because they can glance at their wrist. The app can also use the phone's location to find nearby businesses, which a website or paper system can't do as naturally

Feedback (What changed after our presentation): Our draft was for any walk-in business, with restaurants kept as a stretch goal because party size would affect the FCFS (First-Come-First-Serve) logic. We got feedback on three points and revised the plan:

- **Include restaurants:** NextIn now includes restaurants in the MVP, as party size and table availability make the queue logic more interesting than a plain first-come line
- **Recommendations in MVP:** Similar nearby business recommendations moved from future scope to a feature in our MVP
- **Queue logic:** We dropped the plain FCFS-only plan. Now restaurants can configure their own seating rules, and all other businesses use FCFS

*We also added Wear OS support so customers can check their queue from a smartwatch

2. Intended Users and Value

User	Who they are	What they can accomplish
Customers	People without reservations at businesses ranging from hair studios to restaurants, either alone or in a group (1-8 people at restaurants)	Browse nearby businesses and wait times, join a queue, track their position on phone or watch, get notified when their turn is near, leave if plans change, and compare other nearby businesses
Business staff	Hosts or employees of the businesses who manage the line or seating	Add walk-ins, see who is next, mark parties as seated or removed, and open or close the queue. At restaurants they can also set up tables and seating rules and see which party to seat next

Value to customers: They know roughly how long they will wait, they are free to leave the building, and they have backup options nearby if the line is long

Value to businesses: One shared list for app users and walk-ins, fewer "how much longer?" questions at the front desk, and (at restaurants) fewer tables sitting empty because the next party in line is too large

Assumptions we are making about users:

- Participating businesses have a staff member who will keep the queue updated during service
- Customers have an Android phone or watch with internet access and are willing to allow notifications so they know when their turn is near
- Location is optional. Customers who share it will see nearby businesses sorted by distance, and customers who don't can still join a queue and track their position
- Customers don’t need to create a full account just to join a queue, so we will use anonymous sign-in for them and email sign-in for staff
- Customers must know that wait times are estimates and that the order is approximate, since seating at restaurants depends on party size and open tables
- For the course, we will test with sample businesses that we set up ourselves, since we can't onboard real ones

3. Core User Experience and Initial Scope

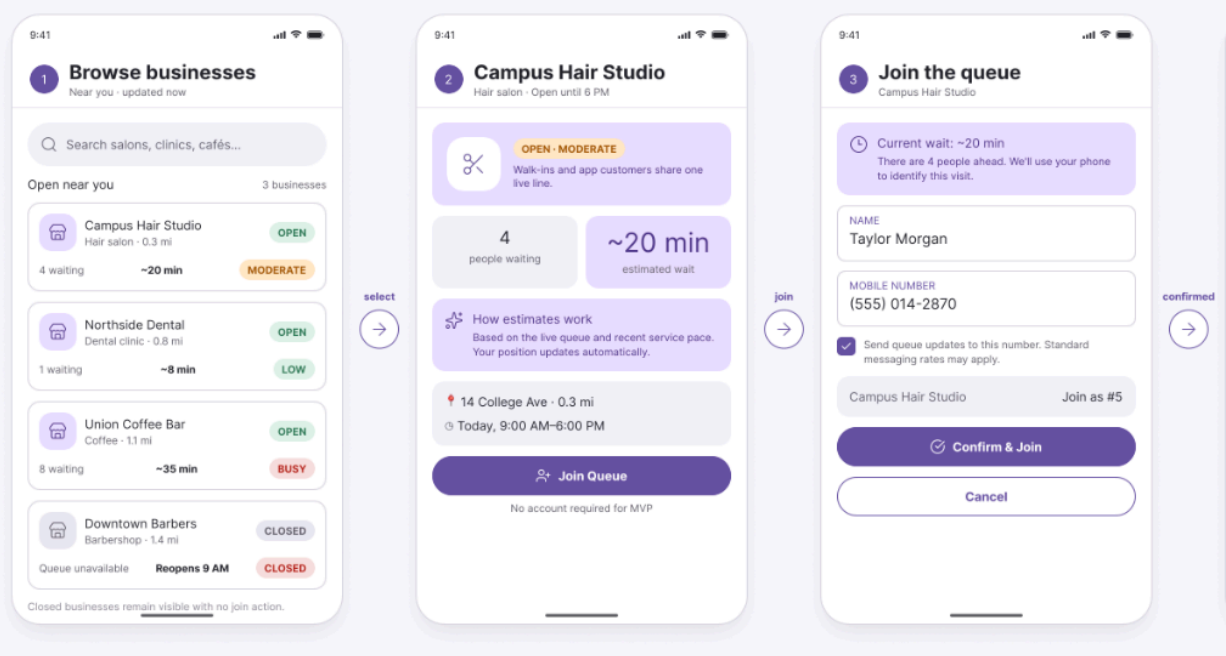
Primary screens

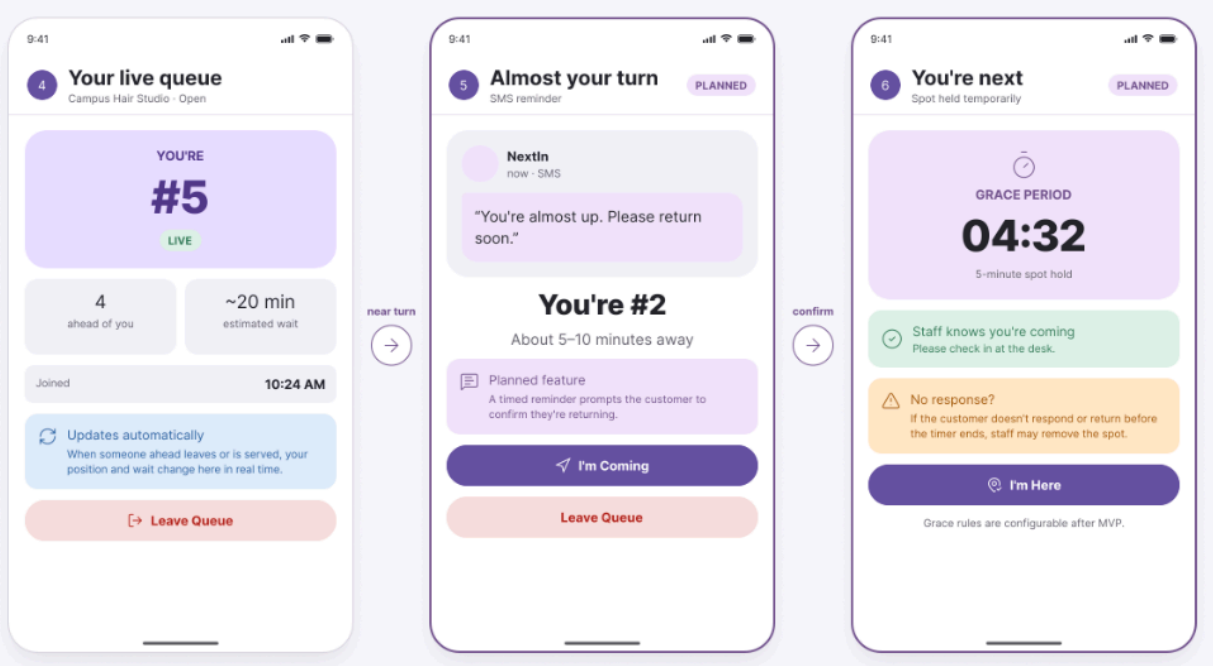
Customers: Can browse participating businesses, search and filter by location, and see business details, including opening status and estimated wait times. They can join a queue by entering their name and party size (at restaurants), track their live position, leave the queue, and confirm when they're called using the "I'm Here" button. If the wait is too long, they can check nearby recommendations. On Wear OS, they can see their business name, position, and estimated wait, with an option to refresh

Business staff: Staff can sign in, view and manage the live queue, add walk-ins, call the next party, and mark customers as seated or removed. The dashboard shows the number of people waiting, estimated wait, and parties seated that day. At restaurants, staff can also set table sizes and seating rules, mark tables open, and see which party is suggested next. Staff can open or close the queue and see whether called parties have confirmed their arrival

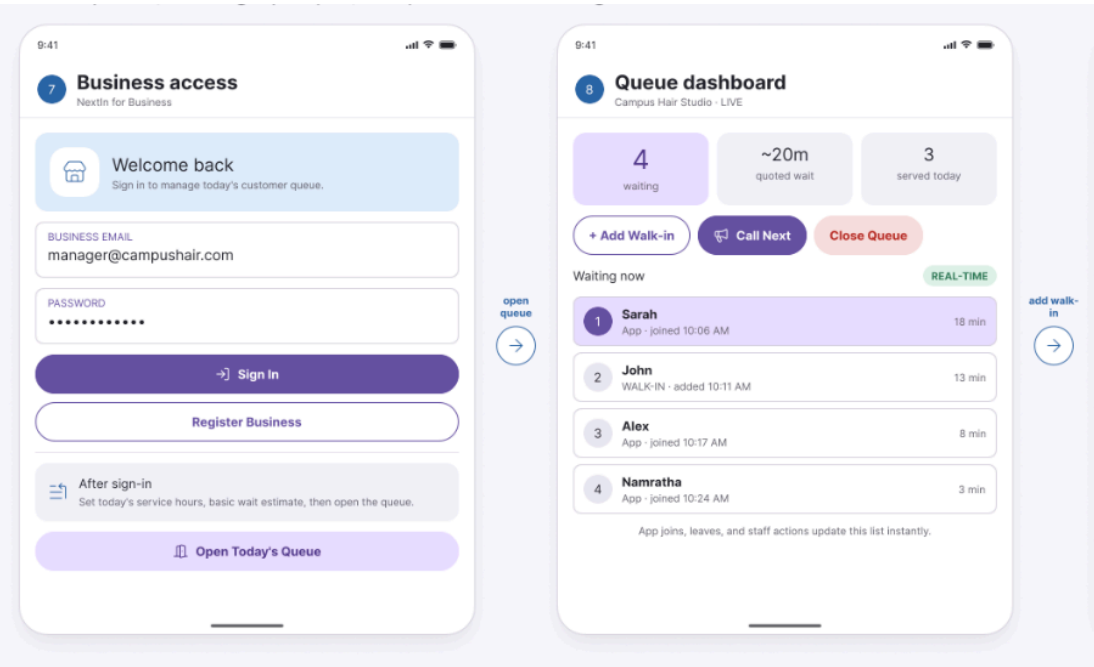
Primary user journeys

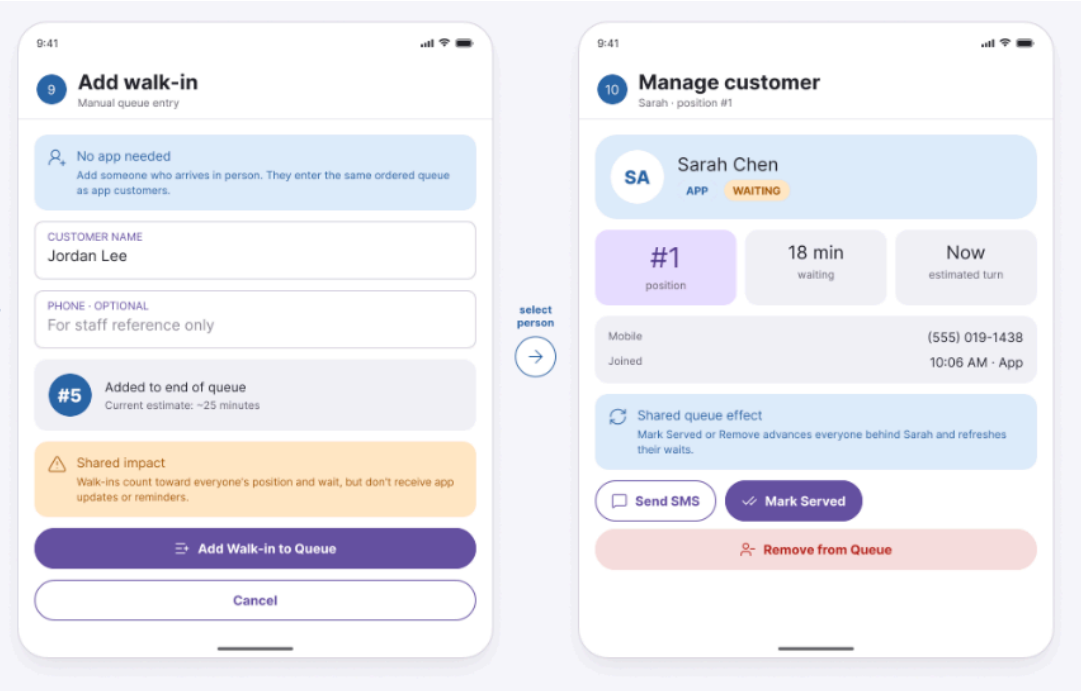
Customers: Browse businesses → Business details + wait → Join queue (name, party size at restaurants) → Live queue status (phone or Wear OS) + notification when turn is near → called → grace period countdown → I'm Here → Seated OR Leave queue → If the wait is long (location on): Recommendations → Compare distance, type, wait → Pick another business





Business Staff: Login → Queue dashboard → (Restaurants: set tables and seating rules) → Add walk-ins / manage parties → (Restaurants: mark a table open) → See suggested next party → Call Next (starts the grace period) → Mark seated (or override) → Shared queue updates





Business-defined queue logic: We don't want to write a different algorithm for every business. Every business apart from restaurants or eateries will follow FCFS. Restaurants will simply fill in a few settings and the app will apply one shared algorithm using them. That way restaurants define their own rules without any custom code. Each business has a business type (restaurant or other), which decides which settings and screens it gets

Setting	Applies to	What the business chooses
Business type	All	Restaurant or other
Seating mode	Restaurants	FCFS (strict arrival order) or Table-fit (earliest party that fits an open table)
Table inventory	Restaurants	Table sizes and how many of each, for example 4 tables for 2, 3 tables for 4, 1 table for 8

Max party size	Restaurants	Largest party accepted through the app
Skip limit	Restaurants (Table-fit)	How many times a party can be passed over before getting priority for the next table that fits their party size. This helps prevent larger parties from being skipped repeatedly
Average service time	All	Minutes per customer (or per party-size group at restaurants), used for wait estimates
Grace period length	All	Minutes a called party has to arrive before staff can remove them (default 5 mins)

How the suggestion works: For most businesses, the app follows FCFS and suggests the first customer in line when staff are ready for the next one. At restaurants, when a staff member marks a table open, the app goes through the waiting list in arrival order. In FCFS mode it suggests the first party. In Table-fit mode it suggests the earliest party that fits that table, unless an earlier party has already reached the skip limit, in which case that party is suggested once a fitting table opens. Staff always make the final decision and can override the suggestion.

Example: A party of 6 is first but a table for 2 opens. In Table-fit mode the app suggests the earliest party of 1-2 and adds one to the skip count of the party of 6. After the skip limit is reached, the party of 6 gets priority for the next table that fits

Wait estimate calculation: For most businesses: estimated wait = (people ahead) x (average service time). For restaurants: estimated wait = (waiting parties ahead that could use the same tables) x (average dining time) / (number of tables that fit this party). These are only rough estimates. We will label them that way in our UI and compare them to our test scenarios

Every party gets an arrival timestamp so the order is consistent and testable. Queue positions are shown as an approximate order, since Table-fit seating can change who goes next

Business setup: For the course, we will set up sample businesses (with their type, settings, and staff accounts) ahead of time, so staff only sign in and manage the queue. A full business sign-up flow is not part of our scope

MVP features, priorities, and acceptance criteria

Priority	User story	Acceptance criteria
P0	As a customer, I want to browse businesses and see estimated waits	The app lists participating businesses with open/closed status and an approximate wait. It works with or without location. Loading, empty, and error states are shown
P0	As a customer, I want to join a queue	I can enter a name and (at restaurants) a party size, join an open queue, and see my queue entry. Empty names, a party size of 0, and sizes above the business's limit are rejected with a message. A customer can only hold one active entry at a time
P0	As a customer, I want to track or leave my queue	My status updates when staff make changes. I can leave the queue, and other parties are not affected. My active entry survives rotation and returning to the app
P0	As business staff, I want to manage waiting parties	A signed-in staff member can add walk-ins, view the list, see the suggested next customer, call the next party, and mark parties as seated or removed. Only that business's staff can do this
P0	As restaurant staff, I want to configure seating rules and mark tables open	Staff can set table sizes, choose FCFS or Table-fit, and set the skip limit. Marking a table open suggests the next party, and changing the rule changes which party is suggested
P0	As a customer, I want recommendations for other nearby businesses	With location permission granted, the app shows nearby businesses by distance and type or cuisine. NextIn businesses show their wait and others show "wait unavailable." If the API fails, a clear error and cached results appear. If permission is denied, the recommendations screen explains that location is needed, and the user can still browse and track their queue

P0	As a customer, when I'm called, I see a countdown and can tap I'm Here	The countdown runs from the time staff call me, it survives rotation and backgrounding, and tapping I'm Here tells staff I'm coming
P0	As business staff, I want to know when a called party hasn't responded	When the grace period ends with no response, the party is marked expired, and staff can remove them or keep them
P1	As a customer, I want to be notified when my turn is near	After I give notification permission, I get a notification when my party is next or has been called. If I deny permission, I can still see my status in the app. Notification says how long they have to arrive
P1	As a customer, I want to check my queue on a smartwatch	A paired Wear OS watch shows the business's name, position, and estimated wait, and has a refresh option
P1	As business staff, I want to open and close my queue	Closing the queue stops new joins, and customers see "closed" on the business's page

Future scope (desirable features that can be deferred)

SMS messages through a third-party service (this would need us to collect phone numbers), a "time to head back" alert that compares travel time to the remaining wait, and location-aware reminders

Explicit non-goals

For this project we will not build:

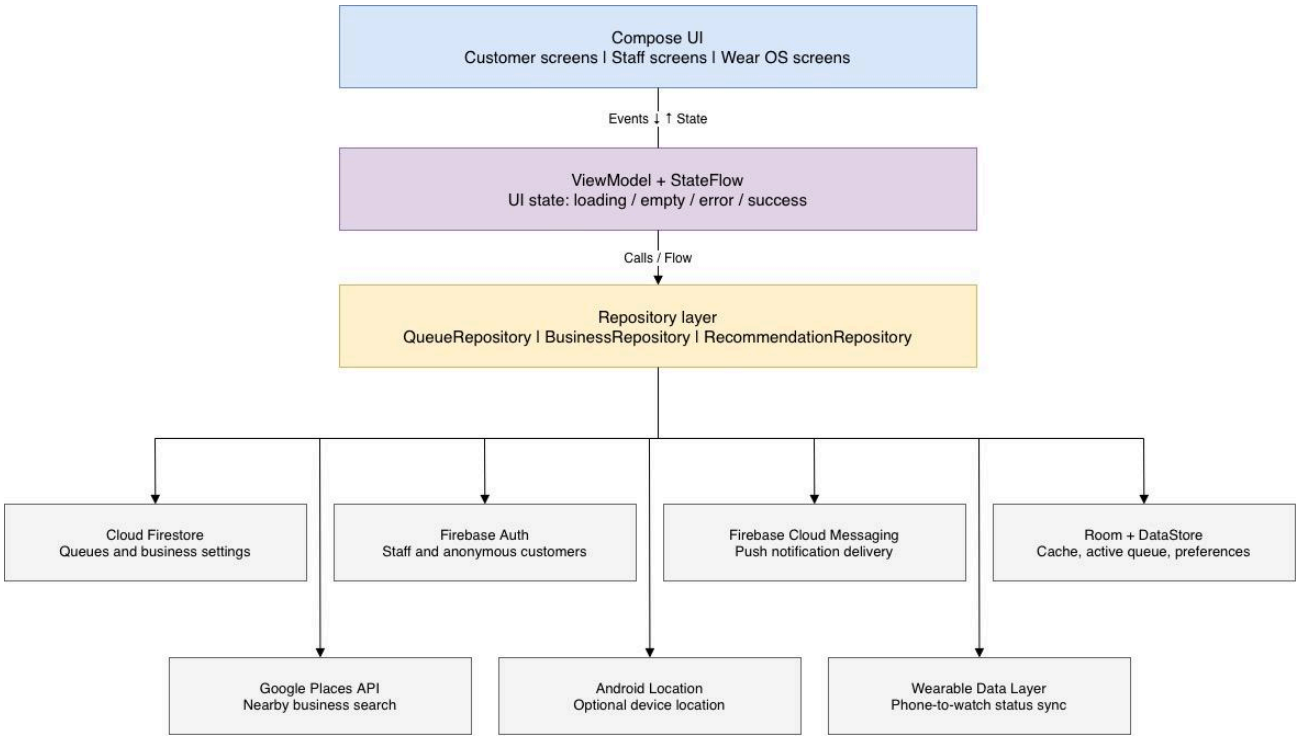
- reservations for specific future times
- food ordering or payments

- detailed floor plans or table maps
- machine-learning wait predictions
- SMS messaging or storing customer phone numbers
- a full business sign-up flow, since sample businesses are set up by us

4. Preliminary Technical Approach

We will build NextIn with Kotlin, the Android SDK, and Jetpack Compose only, with no XML views or Fragments. Navigation Compose handles screen flow, using one navigation graph for customers and one for staff, with the right graph chosen at launch

Architecture and data flow



Example data flow: a customer joins a queue

1. The customer fills in name and party size (at restaurants). The ViewModel validates the input
2. The ViewModel calls QueueRepository.joinQueue() in a coroutine, so the UI is never blocked

3. The repository runs a Firestore transaction that checks the queue is open and the party size is allowed, then creates the entry with a server timestamp
4. The entry's ID is saved in Room as the customer's active queue
5. A Firestore snapshot listener receives queue updates, which the repository exposes as a Flow. The ViewModel uses these updates to calculate or retrieve the customer's position, updates StateFlow, and the UI recomposes
6. The phone forwards the status to the watch through the Data Layer
7. When the party is next or called, a Firebase Cloud Function triggers an FCM push notification, if the customer has allowed notifications

Technical decisions

UI state and lifecycle: Each screen will have a ViewModel exposing StateFlow, collected in a lifecycle-aware way. State will be hoisted out of composables, and saved state plus Room will keep the customer's active queue entry across rotation, backgrounding, and process death. We'll use Kotlin coroutines for network and database work, with loading, empty, success, and error states to handle failures without crashing

Firebase: Cloud Firestore will store businesses, queue entries, and settings. Snapshot listeners will provide live updates, while transactions will handle actions that could happen at the same time, such as customers joining or staff seating someone while another customer leaves. Firebase Authentication will use email and password for staff and anonymous sign-in for customers

External API and local storage: Google Places API will provide nearby business information, including names, categories or cuisine, locations, and distance. Room will cache Places results and store the customer's active queue reference, while DataStore will store preferences like search radius and the last cuisine filter. Firestore will remain the source of truth for live queues, so cached data won't be shown as current when offline

Location and recommendations: Location will be requested at runtime through the fused location provider and used only while the app is in the foreground. It will be optional, so customers can still browse, join, and track queues without it. Recommendations will use distance, business type or cuisine, and available wait estimates to suggest alternatives. Businesses outside NextIn will still appear with "wait unavailable." We won't store location history

Notifications and Wear OS: Firebase Cloud Messaging will send push notifications when a party is next or called, triggered by a Firebase Cloud Function. We'll request notification permission on Android 13 and above. Walk-ins will be called by staff in person, and SMS will remain outside our scope. The Wear OS app will receive queue status from the phone through the Wearable Data Layer API, without requiring a separate login. The watch will show a read-only status screen, with refresh if we can get it working

Queue logic and grace period: The app will use one shared queue algorithm based on business type and settings. Non-restaurants will follow FCFS, while restaurants can choose FCFS or Table-fit with a skip limit. The grace period countdown will use the server's calledAt timestamp so it stays consistent after rotation or backgrounding. Staff will confirm removal when a party doesn't respond

Accessibility and privacy: We'll use readable text, sufficient contrast, content descriptions, 48dp touch targets, and font scaling. We'll test on different screen sizes and with rotation. We'll only collect the information needed for queues and notifications, with no phone numbers or location history. Firebase Authentication and security rules will restrict access to customer and business data

Planned data model

Where	Data
Firestore: business/{id}	name, address, latitude, longitude, business type, cuisine, placeId, queueOpen, staff user IDs, settings (seating mode, skip limit, max party size, average service time), table inventory (restaurants only)
Firestore: business/{id}/queue/{entryId}	name, party size, source (app or walk-in), status (waiting, called, seated, left, removed, expired), joinedAt timestamp, skip count, customer UID, notification token, calledAt timestamp, confirmedArrival (true/false)
Room: cached businesses	placeId, name, business type, latitude, longitude, last fetched time
Room: active queue	business ID, entry ID, business name, join time
DataStore	search radius, last cuisine filter

Security rules plan: Customers can create queue entries under their own UID and only read or leave their own entries. Only a business's listed staff can view the full queue, change settings, and seat or remove parties. To show queue positions and estimated waits without exposing other customers' names or details, we'll provide customers with limited queue summaries. We'll test these rules using the Firebase emulator

Testing plan:

- Unit tests for queue ordering, FCFS and Table-fit suggestions (including the skip limit), both wait estimate formulas, input validation, and recommendation scoring
- Compose UI test for the main customer flow: open a business, join, and see the status screen
- Manual testing of rotation, backgrounding, offline mode, denied location and notification permissions, font scaling, and the watch
- Unit tests for grace period expiry and “I’m Here” confirmation

Areas we still need to research:

- Firestore security rules for the staff and customer roles
- Places API cost, quotas, and key restrictions
- Queue synchronization across several devices and how transactions behave under load
- How to send notifications (Firebase Cloud Functions and the free tier limits) and how reliable they are when the app is in the background
- How often the Wear OS app can realistically refresh, and whether to use a tile or an app screen

5. Team and Development Plan

Roles and responsibilities: Namratha will be responsible for customer screens and navigation, live queue status and lifecycle state, Room and DataStore, recommendations UI, location integration, the Wear OS app, and Compose UI testing. Suchir will be responsible for staff screens, the Firebase data model, authentication and security rules, business types and seating rule settings, queue logic, and staff queue operations. Both of us will collaborate on architecture decisions, Firestore, Google Places API and notification integration, code reviews, queue logic unit tests, debugging, the README, final paper, and final demo. Our roles may shift based on what turns out to be hard. Both of us will understand and be able to explain the whole app

Communication and Git workflow:

We talk through WhatsApp, Google Meet, and in person, with at least two planned check-ins each week plus extra discussions if needed. We will work in weekly iterations and show working

features at each milestone. Tasks will be tracked in GitHub Issues on a shared project board with To Do, In Progress, and Done columns. Every change will follow this path:

GitHub Issue → Feature branch → Pull request → Review by the other person → Merge to main

We will commit often with clear messages so both of our contributions show across the whole semester. Any change to the agreed MVP is written down in an issue or on the board

Weekly Scrum Master Rotation: The scrum master will alternate between Namratha and Suchir each week. Namratha will take the role during Oct 20-26, Nov 3-9, Nov 17-23, and Dec 1-7. Suchir will take the role during Oct 27-Nov 2, Nov 10-16, and Nov 24-30. The scrum master will run the check-ins, update the backlog, and bring up blockers

Working agreement:

- Tell each other early when we are stuck or behind
- Keep our GitHub issues up to date
- Review each other's pull requests before merging anything major
- Talk before changing the agreed MVP
- Both of us contribute all semester, take part in testing, and can explain any part of the code and design
- If one of us is unavailable or falls behind, we will tell the other

Milestone plan:

Dates	Planned work
Oct 20 (Proposal)	Final scope, seating rules, user stories, architecture, and backlog
Oct 21-27	Android project, navigation, Firebase setup, initial security rules, and data models
Oct 28-Nov 3	Customer business list, details, and join screens, staff login and dashboard UI

Nov 4-10	Connect UI to Firestore, join, leave, and serve working end to end, live updates
Nov 10 (Project Update 1)	Demo the vertical slice (UI, state, and Firestore) and explain the architecture and blockers
Nov 11-17	Business types and restaurant seating rules, wait estimates, Room and DataStore, Places API recommendations
Nov 18-24	Optional location, notifications, Wear OS status screen, security rule testing, grace period, and testing the main flows
Nov 24 (Project Update 2)	Feature-complete beta with early tests, known issues, and release plan
Nov 25-Dec 1	Bug fixes, rotation and lifecycle tests, accessibility pass, offline and error states
Dec 2-7	Final testing, README, 1,000-1,500 word paper, architecture diagram, presentation, backup video
Dec 8 (Final submission)	Submit the tagged final repository and present in class

*If we fall behind, we'll cut watch refresh, staff open/close, and extra recommendation filters first. We can simplify the restaurant Table-fit settings if needed, but we'll keep basic restaurant seating logic and nearby recommendations in the MVP

Initial prioritized backlog

ID	Task	Priority	Owner	Target
B1	Create Android project, Compose theme, and navigation graphs	P0	Both	Oct 27
B2	Set up Firebase (Firestore, Auth), initial security rules, and a sample config file	P0	Suchir	Oct 27
B3	Define business, queue entry, and settings data models (with business type)	P0	Suchir	Oct 27
B4	Browse businesses and business details screens	P0	Namratha	Nov 3
B5	Join queue form with input validation (name and party size at restaurants)	P0	Namratha	Nov 3
B6	Staff login and queue dashboard UI	P0	Suchir	Nov 3
B7	Join, leave, and serve with Firestore transactions and live listeners (FCFS)	P0	Both	Nov 10
B8	Live queue status screen with saved state	P0	Namratha	Nov 10

B9	Staff add walk-in, seat, and remove	P0	Suchir	Nov 10
B10	Restaurant settings screen, mark table open, Table-fit logic, skip limit, unit tests	P0	Suchir	Nov 17
B11	Wait estimate calculation (both formulas) and unit tests	P0	Both	Nov 17
B12	Room cache and DataStore preferences	P0	Namratha	Nov 17
B13	Places API repository, recommendation scoring, and recommendations UI	P0	Namratha	Nov 17
B14	Optional location permission flow, distance sorting, and no-location behavior	P0	Namratha	Nov 24
B15	Test and refine Firestore security rules for staff and customers using the Firebase emulator	P0	Suchir	Nov 24
B16	Notifications (permission, Firebase Cloud Messaging, Cloud Function)	P1	Both	Nov 24
B17	Grace period countdown, I’m Here button, and expiry on the staff dashboard	P0	Both	Nov 24
B18	Wear OS status app and phone to watch sync	P1	Namratha	Nov 24
B19	Staff open/close queue	P1	Suchir	Nov 24

B20	Compose UI test for the join flow and accessibility pass	P0	Namratha	Dec 1
B21	Offline, empty, and error states, bug fixes	P0	Both	Dec 1
B22	README, paper, architecture diagram, presentation, backup video	P0	Both	Dec 7

Risks and mitigations

Type	Risk	Mitigation
Technical	Simultaneous joins or staff actions create inconsistent queue order	Firestore transactions, stable entry IDs, arrival timestamps, and unit tests
Technical	Table-fit seating gets too complicated	Only two modes plus a few settings, no custom algorithms. Test with fixed example scenarios
Technical	Wear OS sync is hard to get working	Finish the phone flow first. The watch is P1 and starts as a read-only status screen
Technical	Firestore rules are wrong and expose data or block valid actions	Study the docs early and test rules with the Firebase emulator

Technical	Notifications are unreliable or need a paid Firebase plan	Check the free tier early. If sending from a Cloud Function is not possible, fall back to in-app status and local notifications while the app is running
Usability	Estimated waits are inaccurate	Label them as estimates, use business-set averages, and update them as the queue changes
Usability	Customers don't understand why someone behind them was seated first	Explain that seating depends on party size and tables, and only show an approximate order
Usability	A customer joins remotely and never shows up	Staff can remove parties, and each customer can hold only one active entry. A grace period after being called, with staff removing expired parties
Usability	Customers deny location or notifications	Both are optional. The app still lets them browse, join, and track their position, and explains what they would miss
Privacy/security	Names and location are sensitive	Collect only what we need, do not collect phone numbers, store no location history, use Firebase Auth and security rules, and keep keys out of Git
External API	Places API has quotas, costs, or outages	Check limits early, restrict the key, cache results in Room, and show cached or fallback business listings with an error message
Schedule	Recommendations, notifications, and Wear OS add more work for two people	Build the phone queue first, then recommendations, then notifications and the watch. Cut watch extras before core features

Schedule	Supporting two kinds of queue logic (FCFS and restaurant Table-fit) adds work	Build FCFS end to end first for Update 1. Add business types and Table-fit after

6. Generative AI Disclosure

ChatGPT was used to brainstorm project ideas and app names, discuss limitations to help define scope, review the project requirements, and assess technical decisions and risks. Figma was used to design the early UI screens. We reviewed every scope and technical decision. We will document any AI help used during implementation in the README and the final paper

7. References

1. Android Developers. "Jetpack Compose." Android Developers.
2. Android Developers. "Navigation with Compose." Android Developers.
3. Android Developers. "ViewModel," "StateFlow," and "Kotlin coroutines on Android." Android Developers.
4. Android Developers. "Room" and "DataStore." Android Developers.
5. Android Developers. "Request location permissions" and "Fused location provider." Android Developers.
6. Android Developers. "Wear OS" and "Compose for Wear OS," "Data Layer API." Android Developers.
7. Google Firebase. "Cloud Firestore." Firebase Documentation.
8. Google Firebase. "Firebase Authentication." Firebase Documentation.
9. Google Firebase. "Firestore Security Rules." Firebase Documentation.
10. Google Maps Platform. "Places API." Google for Developers.